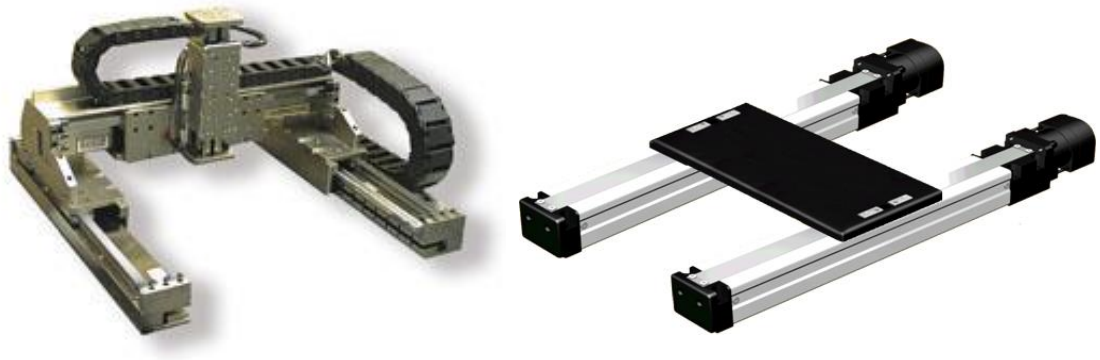
# 겐트리(Gantry) 제어

## 1. 겐트리(Gantry)?

두 개의 문형탑을 연결하는 골조 구조의 총칭이라는 의미로 두 개의 모터 축이 강성 기구물에 의해 연결되어 있으면서 두 개의 축이 하나의 축처럼 제어되는 시스템을 말한다. 한 축이 다른 한 축에 종속되어 제어되므로 주축이 되는 축을 일반적으로 Master축이라 하고 Master에 종속되어 구동되는 축을 Slave축이라 부른다.

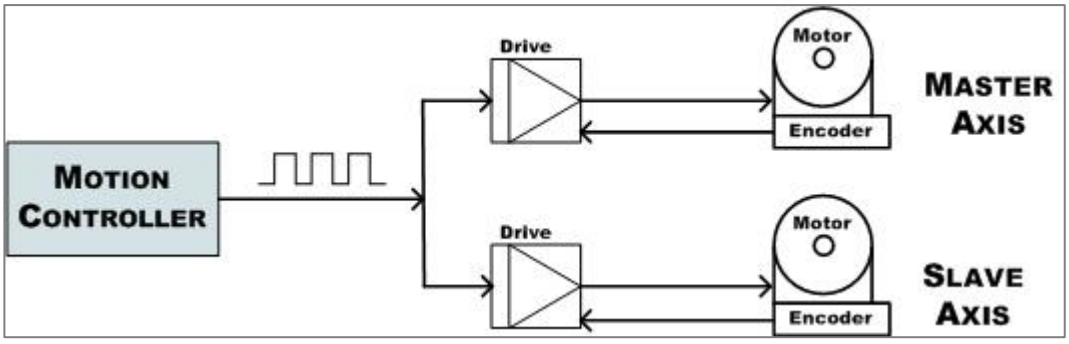


[그림 1] Gantry stage

## 2. 겐트리 제어

### 2.1 제어 방법

당사의 AXL 라이브러리가 제공하는 겐트리 제어 기능은 펄스 출력 타입 제품 사용 시 두 축에 전달되는 지령을 동기화하여 제어하고, 두 축 중 한 축이 Alarm 발생 또는 End Limit 센서에 의한 정지를 할 경우 동시 정지하도록 되어있다. 일반적으로 리니어 서보모터 2축을 사용하는 겐트리 시스템에서 두 겐트리 간의 기구적인 편차가 일정 값 이상 존재하면 서보모터에 과부하가 발생하여 알람이 발생할 여지가 있기 때문에 두 축에 전달되는 지령을 완전히 동기 시키는 방법이 사용된다.




[그림 2] 펄스 출력 타입 제품의 겐트리 시스템 제어

## 2.2 겐트리 제어 함수

두 개의 축을 하나의 축처럼 제어하므로 겐트리 설정 이후에는 Master축에만 지령을 내리면 두 축이 제어 가능하다. 이 때, Slave축에 구동 명령이나 정지 명령을 내려도 모두 무시된다.

| Function | Description |
|---|---|
| AxmGantrySetEnable | 두 축을 겐트리 제어로 설정하면 해당 축의 Master와 Slave축은 동기되어 구동된다. 만약 겐트리 설정 이 후 Slave축에 구동명령이나 정지 명령 등을 내리면 모두 무시 된다. |
| AxmGantryGetEnable | 겐트리 제어로 설정할 때 설정 한 파라메타를 반환한다. |
| AxmGantrySetDisable | 겐트리 제어를 해제한다. |

겐트리 설정과 관계가 있는 함수는 위의 3가지 함수가 있으며 그 중 AxmGantrySetEnable 함수의 사용으로 Master축과 Slave축이 동기화된다.

**Function**

AxmGantrySetEnable

**Prototype**

DWORD AxmGantrySetEnable (long lMasterAxisNo, long lSlaveAxisNo, DWORD dwSlHomeUse, double dSlOffset, double dSlOffsetRange);

**Input/Output**

| Name | in/out | Format | Init Value | Explanation |
|---|---|---|---|---|
| MasterAxisNo | in | long | - | Master축 번호(0 ~ (최대축수 – 1)) |
| SlaveAxisNo | in | long | - | Slave축 번호(0 ~ (최대축수 – 1)) |
| SlHomeUse | In | DWORD | - | Master축과 함께 Slave축도 원점검색을 할 것인지 여부 |
| SlOffset | In | double | - | Master축의 원점센서와 Slave축 원점센서간의 기구적인 오차 값 |
| SlOffsetRange | In | double | - | 원점검색시 Master축의 원점센서와 Slave축 원점센서간에 허용할 최대 오차값 지정 |

**uSlHomeUse**

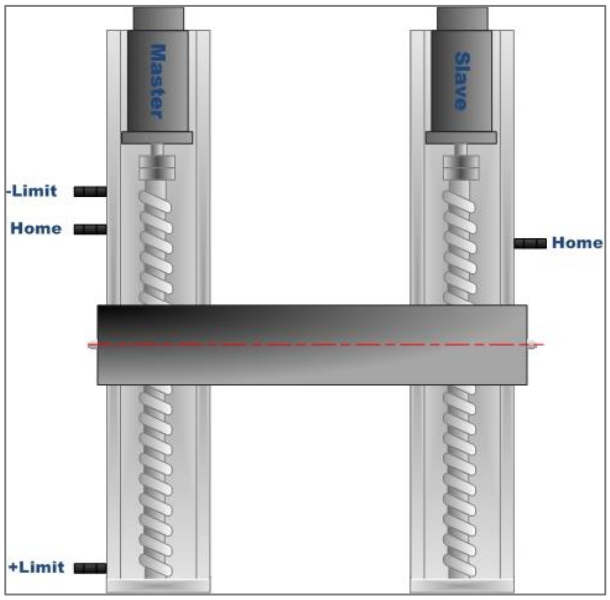
| Value | Explanation |
|---|---|
| 00h | Master축만 원점검색 후 종료(Slave축 원점검색 하지 않음) |
| 01h | Master축, Slave축 원점검색 종료 후 기구물 틀어짐 보정 |
| 02h | Master축 원점검색 후 Slave축 원점검색 완료되면 종료(기구물 틀어짐 보정하지 않음) |

## 2.3 겐트리 시스템 구동

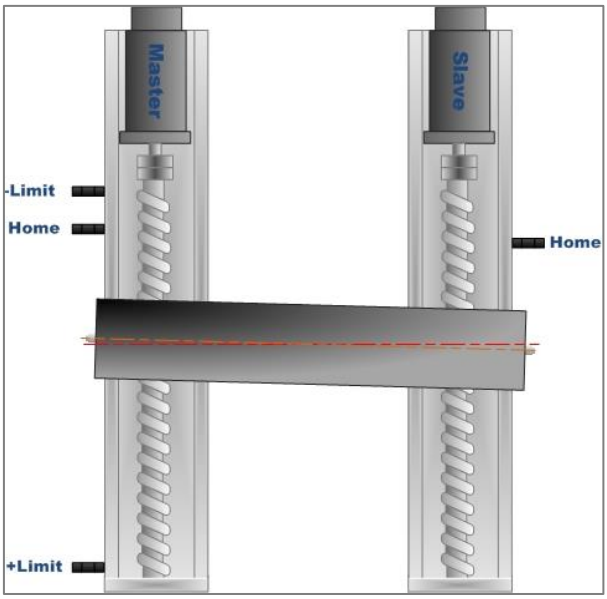
겐트리 시스템을 구동하기에 앞서 두 축의 모션설정을 확인하여야 한다.
전자기어비, Unit/Pulse, 최소속도, 최대속도 등의 설정은 기구물이 동일할 경우 Master축과 Slave축이 동일하게 설정 되어야 한다. 그리고 두 축이 동시에 Servo On이 되어 있는 상황에서 반드시 AxmGantrySetEnable 함수를 실행하여 두 축이 동기 되도록 설정하여야 한다. 만약, 위 함수가 실행되지 않은 상황에서 구동 명령을 내리면 축이 독립적으로 구동하여 기구물에 충격을 줄 수 있다. 그리고 AxmGantrySetEnable 함수를 실행하고 실행된 결과값도 반드시 확인 할 필요가 있다. 위의 설정이 완료되었다면 원점검색을 실시하도록 한다.

## 2.4 겐트리 시스템에서의 원점 검색

겐트리 시스템 구조에서는 기구물의 틀어짐을 보상하기 위해서는 Master, Slave 각 축에 원점센서를 사용하여야 한다. Master축과 Slave축이 완전히 평행하고, 각 축에 연결된 기구물이 수직을 이루도록 기구물을 완벽하게 제작하였다 하더라도 장시간 구동을 하게 되면 기구물은 틀어질 수 밖에 없다. 이 기구적인 틀어짐을 각 축의 원점센서를 사용한 원점구동으로 보정이 가능하다.



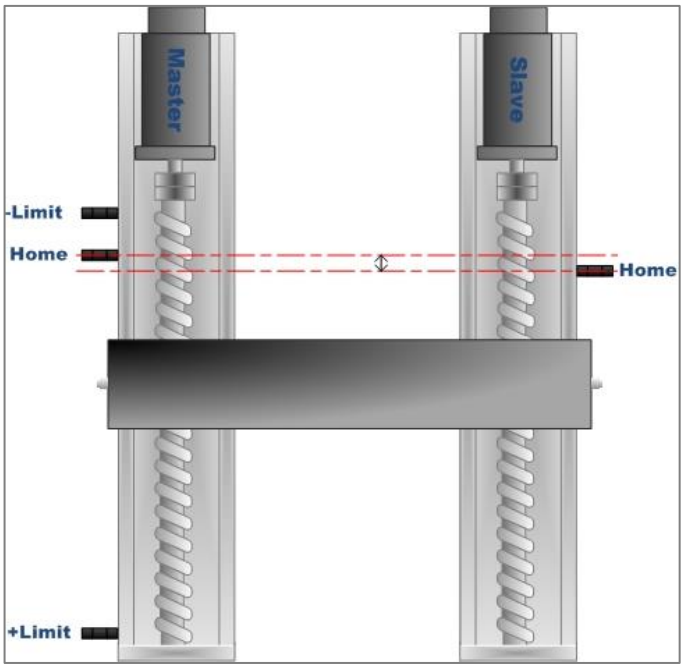

[그림 3] 기구물 제작 시




[그림 4] 기구물 틀어짐

(+), (-)End Limit 센서는 Master축에만 사용하여도 무방하다. (단, Master축과 Slave축의 Stroke이 같을 경우)

원점센서는 겐트리 시스템의 기구물이 틀어졌을 때 바로 잡을 수 있는 기준이 된다. 하지만, 기구물에 원점센서를 설치할 때 오차 없이 완벽하게 평행하도록 설치하는 것은 매우 힘든 작업이다. 그 원점센서 간의 오차를 보상해주는 것이 AxmGantrySetEanble 함수의 네 번째 매개변수인 SlOffset이다. 이 함수의 네 번째 매개변수에 원점센서 간의 오차를 설정하면 원점검색의 마지막 단계인 기구 틀어짐을 보정할 때 반영하게 된다.




[그림 5] 원점센서 오차

AxmGantrySetEnalbe 함수의 세 번째 매개변수 SlHomeUse의 설정에 따라 원점검색은 3가지 단계로 구분된다.

SlHomeUse = 0, Master축만 원점검색을 실시하고 완료되면 종료한다.

SlHomeUse = 1, Master축 원점검색 후 Slave축 원점검색을 실시하고 완료되면 원점센서의 오차를 보상하고 종료한다.

SlHomeUse = 2, Master축 원점검색 후 Slave축 원점검색을 실시하고 완료되면 종료한다.

```
// 0축과 1축을 이용하여 겐트리 구동을 시작한다.
long      lMasterAxisNo = 0;
long      lSlaveAxisNo  = 1;
DWORD dwGantryOn  = 0;
DWORD dwSlHomeUse= 2;            // Master축 원점검색 후 Slave축 원점검색
double   dSlOffset = 0;           // 마스터와 슬레이브 홈센서들간의 기구적인 Offset값
double   dSlOffsetRange = 10;     // 원점 검색시 마스터 홈센서와 슬레이브 홈센서간의 오차 한계
AxmGantrySetEnable(lMasterAxisNo, lSlaveAxisNo, dwSlHomeUse, dSlOffset, dSlOffsetRange);

// AxmGantrySetEnable로 설정한 값을 반환한다.
AxmGantryGetEnable(lMasterAxisNo, &dwSlHomeUse, &dSlOffset, &dSlORange, &dwGatryOn);
```

AxmHomeSetMethod, AxmHomeSetVel, AxmHomeSetStart 함수를 실행하여 원점검색을 실시한다.

<겐트리 시스템 원점 Set up 하기>

① AxmGantrySetEnable함수의 세 번째 매개변수 SlHomeUse의 값을 2로 설정한다.

② 원점검색 구동을 실시한다.

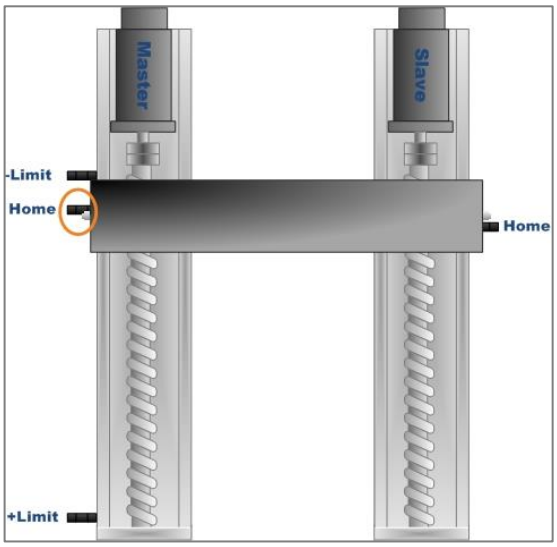
Master축의 원점검색 구동을 실시한다.

Master축의 원점검색 완료.

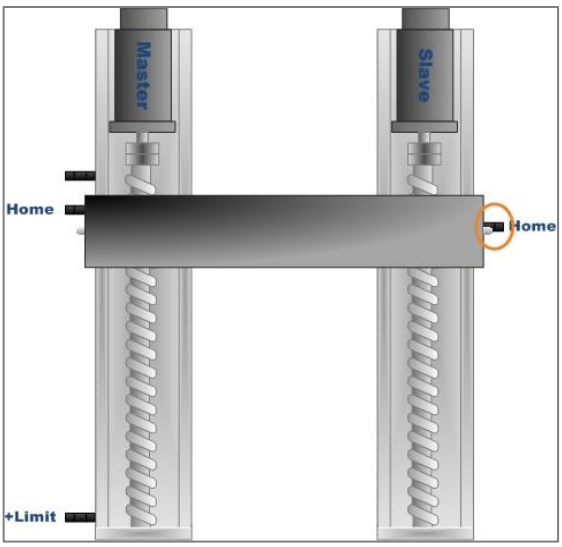
Master축과 Slave축의 Command Position, Actual Position을 0으로 초기화 한다.

Slave축의 원점검색 구동을 실시한다.

Slave축의 원점검색 완료.




[그림 6] Master축 원점검색 완료




[그림 7] Slave축 원점검색 완료

Master축과 Slave축의 Actual Position 0으로 초기화한다.

이때, Master, Slave축의 Command Position에 남아있는 값이 원점센서 오차이다.

③ AxmGantrySetEnable함수의 세 번째 매개변수 SlHomeUse의 값을 1로 설정한다.

④ 앞의 과정에서 확인했던 원점센서의 오차 값에 -1을 곱하여 AxmGantrySetEnable함수의 네 번째 매개변수 dSlOffset의 값으로 설정한다.

⑤ 원점검색 구동을 실시한다.

Master축의 원점검색 구동을 실시한다.

Master축의 원점검색 완료.

Master축과 Slave축의 Command Position, Actual Position을 0으로 초기화 한다.

Slave축의 원점검색 구동을 실시한다.

Slave축의 원점검색 완료.

Master축과 Slave축의 동기가 해제된다.

Master축은 Master축의 0 위치로 이동한다.

Slave축은 dSlOffset의 값만큼 이동한다.

두 축 모두 이동이 완료되면 다시 동기화 된다.

⑥ 원점 Set up 완료.